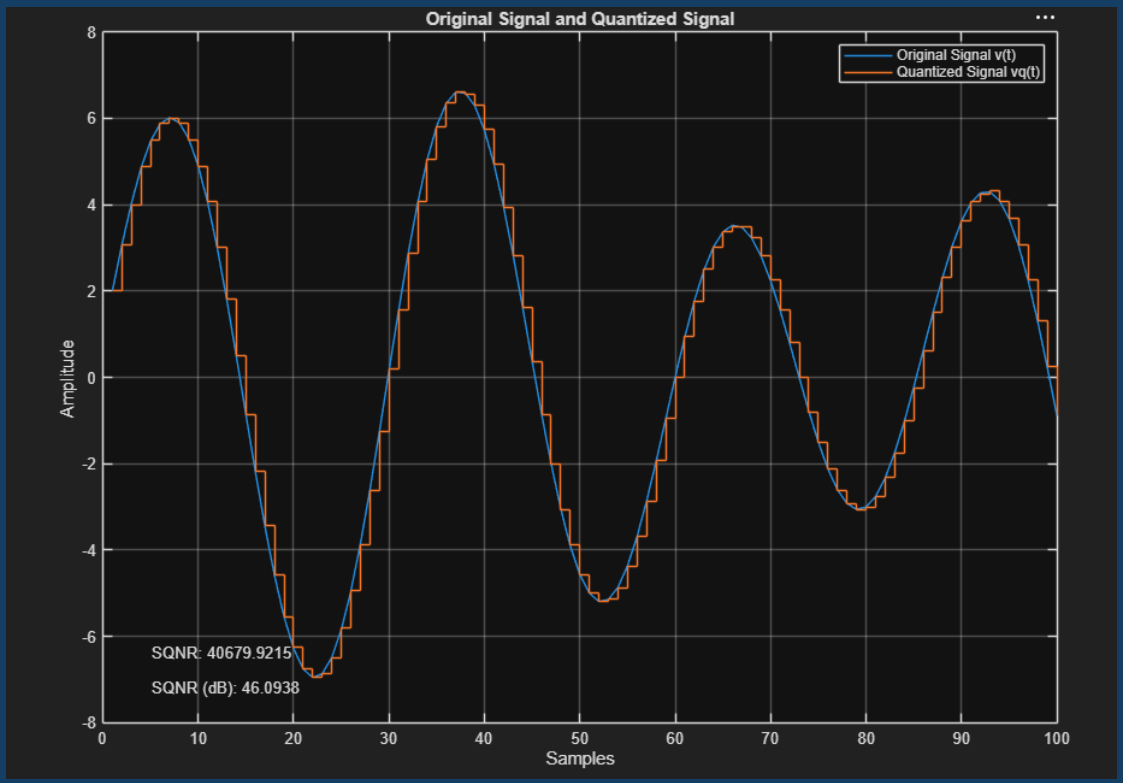
User Generated Analog Signal Data: Calculates the quantized value for each data point using ADC

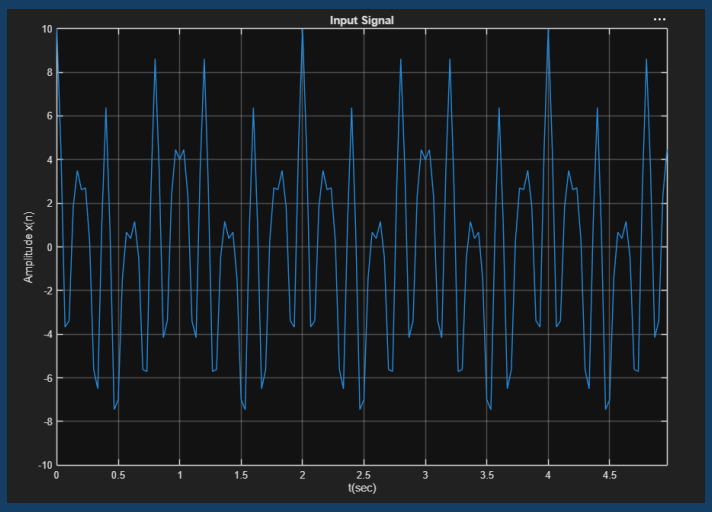
```
% Step 3: Ask user to enter sampled data vector:
sampled_data = input('Enter the sampled data vector: '); % ex inputs: 2*cos(5*pi*(0:99)*0.01)+5*sin(7*pi*(0:99)*0.01)

% Step 4: Ask user to enter ADC information; number of bits, maximum and minimum reference voltages: [N, Vref+, Vref-]:
adc_info = input('Enter ADC info as a vector [N, Vref+, Vref-]: '); % ex inputs: [8, 8, -8]

N = adc_info(1);          % Number of bits:           vector=1=N,     called from user sampled data input
Vref_plus = adc_info(2);  % Maximum reference voltage: vector=2=Vref+, called from user sampled data input
Vref_minus = adc_info(3); % Minimum reference voltage: vector=3=Vref-, called from user sampled data input

% Step 5: Calculate the step number and the step size:
num_steps = 2^N;                                % number of levels (2^N for unsigned, 2^(N-1) for signed)
step_size = (Vref_plus - Vref_minus) / num_steps; % step number,i,corresponding input voltage

% Step 6: Quantize voltage:
vq = Vref_minus + round((sampled_data - Vref_minus) / step_size) * step_size; % input sampled data vector and generate the equivalent quantized value

% Step 7: Plot original sampled data and Quantized data:
figure;
plot(sampled_data);
hold on;
stairs(vq);
xlabel('Samples');
ylabel('Amplitude');
title('Original Signal and Quantized Signal');
legend('Original Signal v(t)', 'Quantized Signal vq(t)');
grid on;

% Step 8: Calculate Signal-to-Quantization Noise Ratio (SQNR):
signal_power = sum(abs(sampled_data.^2));  % measure of how much noise has been introduced in quantized signal by the quantization error'eq'.
eq = sampled_data - vq;                    % quantization error
noise_power = sum(abs(eq.^2));             % measure of how much noise has been introduced in quantized signal by the quantization error'eq'.
SQNR = signal_power / noise_power;         % samples of input and corresponding values of quantization voltages and quantization error values
SQNR_dB = 10 * log10(SQNR);

% Display SQNR values in the plot window:
text(0.05, 0.1, ['SQNR: ', num2str(SQNR)], 'Units', 'normalized', 'HorizontalAlignment', 'left');
text(0.05, 0.05, ['SQNR (dB): ', num2str(SQNR_dB)], 'Units', 'normalized', 'HorizontalAlignment', 'left');

hold off;
```

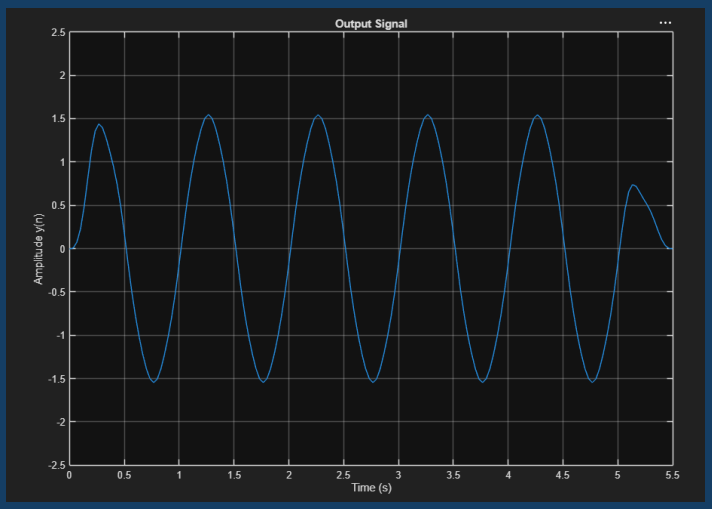


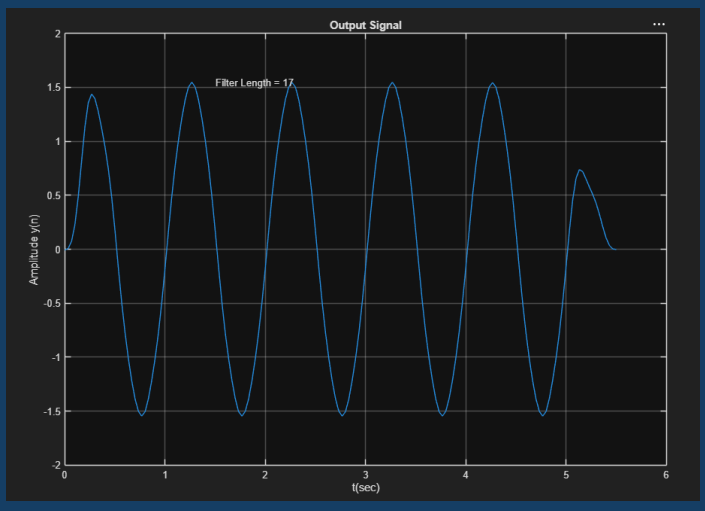
Generates a Compound Sinusoidal Signal: filters using a low-pass FIR filter. The objective is to observe the characteristics of the digital system.

```
% Define parameters
N = 150;              % Number of samples-> length of the input data vector
fmax = 7.5;           % highest frequencies from simply input signal
%fmax = 15;           % highest frequencies from simply input signal
nyq_rate = 2*fmax;    % Sampling rate
fs = 2*nyq_rate;      % twice the Nyquist rate
n = 0:N-1;            % samples
Ts = 1/fs;            % Sampling period
t = n*Ts;             % Time vector
fc = 2;               % Cutoff frequency of the digital lowpass filter in Hz: given

% Step 3: Generate the input signal x(t)
x = 2*cos(2*pi*t) + 5*cos(10*pi*t) + 3*cos(15*pi*t);  % frequency = 1Hz, 5Hz, 7.5Hz
%x = 2*cos(1*pi*t) + 5*cos(5*pi*t) + 3*cos(7.5*pi*t); % simplified signal

% Step 4: Plot the input signal
figure(1);
plot(t,x);
title('Input Signal');
xlabel('t(sec)');
ylabel('Amplitude x(n)');
grid on;
axis([0 max(t) -10 10]);
```

```
% Step 5: Impulse response h(n) of the DSP lowpass filter: "to see clean 1Hz signal"
% test trial of Filter Length:

%M = 6;                              % 13 taps; (initial recommended; minimum)
M = 8;                               % 17 taps; (ok performance taps)
%M = 10;                             % 21 taps; (good performance; maximum)

k = -M:M;                            % Range
a = (sin(2*pi*fc*k/fs)) ./ (pi*k);   % Impulse response calculation formula: number of terms will be odd
a(M+1) = 2*(fc/fs);                  % Impulse response calculation formula: number of terms will be odd
b = 0.54+0.46*cos(pi*k/M);           % The Hamming window: number of terms will be odd
h = a.*b;                            % Impulse response of the DSP system: number of terms will be odd

% Step 6: Calculate the output using digital convolution
y = conv(x,h);                        % Convolution: output signal
n1 = 0:Ts:(length(y)-1)*Ts;           % Time vector for convolution result

% Step 7: Plot the output signal y(n)
figure(2);
plot(n1,y);
title('Output Signal');
xlabel('Time (s)');
ylabel('Amplitude y(n)');
grid on;
hold on;
axis([0 max(n1) -2.5 2.5]);

% Step 8 & 9: Plot with the final output signal
filter_length = 2*M + 1;              % Display length of the FIR filter
figure(3);
plot(n1,y);
title('Output Signal');
xlabel('t(sec)');
ylabel('Amplitude y(n)');
grid on;
hold on;
```


Input Signal
Amplitude x(n)
t(sec)


Output Signal
Amplitude y(n)
Time (s)


Output Signal
Filter Length = 17
Amplitude y(n)
t(sec)